More Than One Parameter

- Separate parameters with _____.
- Arguments are matched to parameters in _____: first to first, second to second.

```
def drawBall(x, y, color):
    Circle(x, y, 25, fill=color)

drawBall(100, 100, 'red')
drawBall(300, 250, 'blue')
```

Order Matters

- `drawBall(100, 100, 'red')` → x is 100, y is 100, color is 'red'.
- `drawBall('red', 100, 100)` → x is 'red' and color is 100. That causes an _____.
- The number of arguments must match the number of parameters, or you get a `TypeError`.
- Tip: give parameters clear names so the order is easy to remember.

Math Operators

Operator	Name	Example	Result
`+`	addition	`7 + 2`	9
`-`	subtraction	`7 - 2`	5
`*`	multiplication	`7 * 2`	14
`/`	division	`7 / 2`	_____
`//`	_____	`7 // 2`	_____
`%`	_____ (remainder)	`7 % 2`	_____
`**`	exponent	`7 ** 2`	49

Parameters + Math

- Parameters can be used in calculations, so one function can draw things at different sizes.

```
def drawSnowman(x, size):
    Circle(x, 300, size, fill='white', border='black')
    Circle(x, 300 - size * 1.6, size * 0.7, fill='white', border='black')
    Circle(x, 300 - size * 2.8, size * 0.5, fill='white', border='black')

drawSnowman(120, 50)
drawSnowman(300, 30)
```

Printing Results

- Commas in `print()` put a _____ between items.
- With commas, you can mix strings and numbers in one `print()`.

```
def forkWeight(boxWeight, forks):
    print('Each fork weighs:', boxWeight / forks, 'lbs')

forkWeight(4, 16)
forkWeight(10, 25)
```

Predict the output:

Order of Operations

- Python follows the same order as math class (PEMDAS):
    - o _____ first
    - o then `**`
    - o then `*`, `/`, `//`, `%` from left to right
    - o then `+` and `-` from left to right
- `2 + 3 * 4` is _____, but `(2 + 3) * 4` is _____.

Check Your Understanding

1. What is `17 // 5`? What is `17 % 5`?

2. What is `2 ** 3`? What is `10 / 4`?

3. What is `10 - 2 * 3 + 1`?

4. `def box(width, height):` is called as `box(50, 20)`. What is `height`?

Try It

Do these in the CMU sandbox. Write down anything you had to fix along the way.

Try It: Utensils (CMU sandbox)

- Define `forkWeight(boxWeight, forks)`.
- It finds the weight of one fork (box weight ÷ number of forks) and prints it in a sentence.
- Test it with a 4 lb box of 16 forks, then two more cases of your own.

Sample output

```
Each fork weighs: 0.25 lbs
```

One thing I had to fix or figure out:

☐ Finished in the CMU sandbox

Adapted from River Hill CompSci, Python – Variables & Math exercises

Try It: Moving Day (CMU sandbox)

- A trolley holds 32 lbs. Define `trips(numBoxes, boxWeight)`.
- Boxes per trip = how many whole boxes fit: use `//`.
- Trips = boxes ÷ boxes per trip, rounded up. Hint: `(numBoxes + perTrip - 1) // perTrip` rounds up.
- Print both numbers.

Sample output

```
trips(24, 8):  4 boxes per trip, 6 trips
trips(14, 12): 2 boxes per trip, 7 trips
```

One thing I had to fix or figure out:

☐ Finished in the CMU sandbox

Adapted from River Hill CompSci, Python – Variables & Math exercises

Try It: Face Factory (CMU sandbox)

- Define `drawFace(x, y, size, color)` that draws a face: a head, two eyes and a mouth.
- Make every part depend on `x`, `y` and `size`, so the face scales.
- Call it at least three times with different arguments.

One thing I had to fix or figure out:

☐ Finished in the CMU sandbox

CMU-style practice